

SECURITY ASSESSMENT

VAPT REPORT

Vulnerability Assessment & Penetration Testing

BlueEra Flutter Mobile Application

Android & iOS | Package ai.bluecs.app | v14.0.19 (277)

Prepared by

Harshit Rajput

Security Engineer

Report Date: 03 July 2026

Assessment Type: White-box Static Application Security Testing (SAST) / Source Code Review

Classification: CONFIDENTIAL

Version: 1.0

— CONFIDENTIAL —

1. Document Control & Disclaimer

Field	Detail
Document Title	BlueEra Mobile Application — Vulnerability Assessment & Penetration Testing Report
Target Application	BlueEra multi-service Flutter super-app (social feed, chat, VoIP/video calling, jobs, food/grocery, medical, hotel, rides, payments)
Platforms	Android (min SDK 26, target SDK 36) & iOS
Application ID	ai.bluecs.app
Version Assessed	14.0.19 (build 277) — pubspec 14.0.18+276
Backend Domains	*.beapp.in, bluecs.in
Prepared By	Harshit Rajput , Security Engineer
Assessment Window	July 2026
Methodology	OWASP MASVS / OWASP Mobile Top 10 (2024), CWE, manual + assisted source code review

Disclaimer. This assessment was performed on a static snapshot of the application source code provided as an archive. It represents a point-in-time review and does not guarantee the absence of all vulnerabilities. Findings that depend on backend behaviour (e.g. server-side authorization, JWT signature validation, rate limiting) could not be confirmed dynamically and are noted as such. All testing was non-destructive. Secrets and tokens disclosed in this report were found already present in the provided source and should be treated as compromised and rotated.

2. Table of Contents

1. Document Control & Disclaimer
2. Table of Contents
3. Executive Summary
4. Scope & Methodology
5. Risk Rating Methodology
6. Findings Summary
7. Detailed Findings (F-01 ... F-20)
8. Positive Security Observations
9. Remediation Roadmap
10. Appendix A — Android Permission Inventory
11. Appendix B — Tools, References & Standards

3. Executive Summary

A white-box security assessment was conducted against the **BlueEra** Flutter mobile application and its native Android/iOS layers. BlueEra is a large multi-service "super-app" (2,472 Dart source files) handling sensitive user data including personal messages, contacts, precise/background location, identity documents, and financial transactions via Razorpay.

The review identified **1 Critical, 5 High, 10 Medium**, and **4 Low/Informational** issues. The most serious problems cluster around three themes:

- **Secret & credential exposure** — Live, still-valid JWT session tokens are committed in source comments, and a developer `local.properties` file containing live Google Maps and Google Gemini (billable) API keys was shipped inside the codebase.
- **Sensitive data leakage to device logs** — The auth Bearer token and full API response bodies (which include the session JWT returned by OTP verification and user PII) are written to the OS log in *release* builds, not just debug.
- **Transport security disabled** — Cleartext HTTP/WS is explicitly enabled on Android (`usesCleartextTraffic=true`) and App Transport Security is fully disabled on iOS (`NSAllowsArbitraryLoads=true`), removing the platform's guarantee against man-in-the-middle attacks and enabling downgrade of the otherwise-correct `wss/https` socket design.

Notably, several areas were found to be implemented **correctly**: Razorpay payment activation is verified server-side (not trusted from the client callback), WebRTC ICE/TURN credentials are fetched per-call from the server rather than hardcoded, auth tokens are held in platform secure storage/Keychain, and deep-link entity IDs are validated against a strict `ObjectId` pattern before navigation. These strengths should be preserved during remediation.

Risk Posture at a Glance

Severity	Count	Distribution
CRITICAL	1	
HIGH	5	
MEDIUM	10	
LOW / INFO	4	

Overall application risk rating: **HIGH** — Immediate remediation of the Critical and High findings is strongly recommended before the next production release, followed by rotation of all exposed secrets.

4. Scope & Methodology

4.1 In Scope

- Flutter/Dart application source (lib/ — 2,472 files)
- Android native layer (AndroidManifest.xml, Gradle build, Kotlin, ProGuard)
- iOS native layer (Info.plist, Swift, entitlements)
- Third-party bundled packages under packages/ and dependency manifest
- Configuration, secrets handling, storage, transport, deep links, payments, IPC

4.2 Out of Scope

- Server-side / API backend (*.beapp.in) — grey-box observations only, no live testing
- Dynamic runtime / instrumented (DAST) testing — not performed on a running device
- Third-party SaaS (Razorpay, Firebase, Meta Audience Network) internal security

4.3 Approach

The assessment followed **OWASP MASVS** (Mobile Application Security Verification Standard) and the **OWASP Mobile Top 10 (2024)**, with weaknesses classified using **MITRE CWE**. Manual source review was augmented with pattern-based static analysis across the codebase covering: insecure data storage, insecure logging, cryptography, transport security, secret management, platform IPC (intents, deep links, method channels), WebView configuration, payment integrity, and authentication/session handling.

5. Risk Rating Methodology

Each finding is rated using a qualitative Likelihood × Impact model aligned with CVSS 3.1 severity bands. Because the assessment targets first-party application code (not a versioned vulnerable third-party component), findings are primarily mapped to **CWE** and **OWASP MASVS/Mobile Top 10** references. A public **CVE** is cited only where a specific known-vulnerable component applies; otherwise "N/A (first-party weakness)" is stated — this reflects standard industry practice for bespoke application code.

Rating	CVSS Band	Definition
CRITICAL	9.0 – 10.0	Trivially exploitable; direct account takeover, credential theft, or full compromise.
HIGH	7.0 – 8.9	Serious impact on confidentiality/integrity; realistic attack path.
MEDIUM	4.0 – 6.9	Meaningful risk requiring specific conditions or partial mitigation present.
LOW	0.1 – 3.9	Limited impact, hardening / defense-in-depth, or requires strong preconditions.

6. Findings Summary

ID	Finding	Severity	CWE	OWASP M-Top10
F-01	Live JWT session tokens hardcoded in source	CRITICAL	CWE-798, 522	M1 / M9
F-02	Auth token & full API responses logged in production	HIGH	CWE-532	M9
F-03	Auth Bearer token printed in production SSE streams	HIGH	CWE-532	M9
F-04	Cleartext traffic enabled (Android)	HIGH	CWE-319	M5
F-05	App Transport Security disabled (iOS)	HIGH	CWE-319	M5
F-06	Developer secrets file shipped with live API keys	HIGH	CWE-798, 312	M1 / M9
F-07	Chat messages & contacts stored unencrypted at rest (Hive)	MEDIUM	CWE-312	M9
F-08	Full device phonebook uploaded raw/unhashed	MEDIUM	CWE-359	M6
F-09	WebView unrestricted JavaScript on untrusted URLs	MEDIUM	CWE-79, 749	M4 / M8
F-10	Deep-link / install-referrer referral code trusted unvalidated	MEDIUM	CWE-20	M4
F-11	Exported broadcast receiver without permission	MEDIUM	CWE-926	M8
F-12	Hardcoded AES key & static IV in bundled CallKit code	MEDIUM	CWE-798, 329, 321	M10
F-13	No certificate/public-key pinning	MEDIUM	CWE-295	M5
F-14	Login OTP written to device logs	MEDIUM	CWE-532	M9
F-15	Over-privileged / broad exported components & permissions	MEDIUM	CWE-926, 250	M8
F-16	Auth token & request bodies logged (debug-gated)	MEDIUM	CWE-532	M9

F-17	Unpinned ("any") dependency version constraints	LOW	CWE-1104	M2
F-18	FCM/VoIP tokens & payloads printed to logs	LOW	CWE-532	M9
F-19	Unsanitized HTML rendered in WebView (JS disabled)	LOW	CWE-79	M4
F-20	Live Razorpay key & Google keys hardcoded in client	LOW	CWE-798	M1

7. Detailed Findings

F-01 — Live JWT Session Tokens Hardcoded in Source Code CRITICAL

CVSS 3.1	9.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
CWE	CWE-798 (Use of Hard-coded Credentials), CWE-522 (Insufficiently Protected Credentials)
CVE	N/A (first-party weakness)
OWASP	M1: Improper Credential Usage / M9: Insecure Data Storage
Location	lib/environment_config.dart:77-79

Description. Two fully-formed, signed HS256 JWTs are committed as comments in the environment configuration file. Base64-decoding the payloads reveals live session material: a GUEST account token (sessionId, user_id, contact_no:0052000000) with expiry exp:1790162672 (≈ mid-2026 — still valid) and a BUSINESS account token.

```
// lib/environment_config.dart
// eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzZXNzaW9uSWQiOiI2OWM2NjhmMGI2...
//   ...account_type":"GUEST", "contact_no":"0052000000"... "exp":1790162672 ...
// .TzABYiQ340Dff8Uw...
// eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfaWQiOiI0NSiX2lkIjoInjY4Zj...
//   ...account_type":"BUSINESS"... "exp":1775042839 ... .mb0a1UqMnad6YpHpZw4C...
```

Impact. Anyone with access to the source (or a decompiled build, since the strings ship in the binary) can replay these Bearer tokens to impersonate the corresponding sessions until expiry, accessing the victim account's data and actions. The tokens also leak the backend's JWT structure and signing scheme.

Remediation.

- **Immediately revoke/invalidate** both tokens server-side and rotate the JWT signing secret.
- Remove the tokens from source *and purge them from git history* (BFG / git filter-repo).
- Never place real credentials in comments or code; use short-lived tokens and a secrets manager for test material.

F-02 — Auth Token & Full API Response Bodies Logged in Production HIGH

CVSS 3.1	7.5 (AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
CWE	CWE-532 (Insertion of Sensitive Information into Log File)
CVE	N/A (first-party weakness)
OWASP	M9: Insecure Data Storage / Insufficient Logging
Location	lib/core/api/apiService/api_base_helper.dart:174-176

Description. The Dio `onResponse` interceptor logs the full JSON response body for every successful request. Unlike the request logging above it, this call sits **outside** the `if (kDebugMode)` guard (which ends at line 139), so it executes in release builds. The OTP-verify endpoint returns the session JWT in its body (`data.token`), meaning the user's live auth token — plus all PII in every response — is written to the OS log in production.

```
// api_base_helper.dart – onResponse (NOT wrapped in kDebugMode)
log('SUCCESS CODE ${response.statusCode} : 🌐 $responseUrl\n${jsonEncode(response.data)}');
```

Impact. Any process able to read device logs (other apps on older Android, ADB, crash/analytics aggregators, MDM log capture) can harvest the session token and personal data, enabling account takeover.

Remediation. Wrap all response logging in `kDebugMode`; redact `Authorization` headers and token fields; adopt a logging utility that strips secrets and is compiled out of release builds.

F-03 — Auth Bearer Token Printed Unconditionally in SSE Streams HIGH

CVSS 3.1	7.1
CWE	CWE-532
CVE	N/A (first-party weakness)
OWASP	M9
Location	lib/features/me/grocery/stream/rider_grocery_stream.dart:24 lib/features/me/medical/stream/rider_medical_stream.dart:24

Description. The rider grocery/medical Server-Sent-Events connectors print the full global auth token to the log on every connect. The label says "length" but the entire token value is interpolated. These calls are **not** gated by `kDebugMode` and therefore ship in release.

```
request.headers.addAll({'Authorization': 'Bearer $authTokenGlobal', ...});  
print("Connecting to: $url");  
print("Auth token length: ${authTokenGlobal}"); // prints the WHOLE token, in release
```

Impact. Same as F-02 — production leakage of a long-lived session credential to device logs.

Remediation. Remove these print statements (or gate behind `kDebugMode` and log only a boolean "token present"). Enforce a lint rule banning print in production code.

F-04 — Cleartext HTTP/WS Traffic Enabled (Android) HIGH

CVSS 3.1	7.4 (AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N)
CWE	CWE-319 (Cleartext Transmission of Sensitive Information)
CVE	N/A (configuration weakness)
OWASP	M5: Insecure Communication
Location	android/app/src/main/AndroidManifest.xml:46

Description. The application element sets `android:usesCleartextTraffic="true"` and ships no network-security-config restricting domains. This permits plaintext HTTP/WS to any host and enables protocol downgrade.

```
<application  
  android:usesCleartextTraffic="true"  
  android:requestLegacyExternalStorage="true"  
  ...>
```

Impact. On a hostile network an attacker can intercept or downgrade connections, exposing tokens, chat, and location data (which are otherwise carried over wss/https).

Remediation. Set `usesCleartextTraffic="false"` and add a `network_security_config.xml` that denies cleartext and whitelists only required TLS domains. Remove `org.apache.http.legacy` if unused.

F-05 — App Transport Security Fully Disabled (iOS) HIGH

CVSS 3.1	7.4
CWE	CWE-319
CVE	N/A (configuration weakness)
OWASP	M5: Insecure Communication
Location	ios/Runner/Info.plist:61-65

Description. `NSAllowsArbitraryLoads=true` disables ATS app-wide, permitting cleartext and weak-TLS connections on iOS.

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <true/>
</dict>
```

Impact. Equivalent to F-04 on iOS — removes the platform MITM/eavesdropping protection. Apple may also reject/scrutinise the app for a blanket ATS exception.

Remediation. Remove the blanket exception. If specific legacy hosts truly need it, scope exceptions with `NSExceptionDomains` per host and require TLS 1.2+.

F-06 — Developer Secrets File Shipped With Live API Keys HIGH

CVSS 3.1	7.5
CWE	CWE-798, CWE-312 (Cleartext Storage of Sensitive Information)
CVE	N/A (first-party weakness)
OWASP	M1 / M9
Location	android/local.properties:6-8

Description. `local.properties` — a per-developer file that must never be committed — is present in the codebase and contains live keys, plus a local filesystem path disclosing the developer's username.

```
MAP_API_KEY=AIzaSyBCQC4611GnFpzFDGiMu1M_UXjCDFigt-Q
GEMINI_API_KEY=AIzaSyCfxq52QNGgnq9d37tztS9VxS6EBjPE21s
ADMOB_APP_ID=ca-app-pub-8886065788013100~3740870813
sdk.dir=/Users/Mr.bhavesh/Library/Android/sdk
```

Impact. The **Gemini API key is billable** — if unrestricted it can be abused to run up cost against the owner's Google Cloud account. Unrestricted Maps keys can likewise be abused. Note the app also embeds the Gemini key into an Android `resValue` (`android/app/build.gradle:68`), making it extractable from the APK.

Remediation. Remove `local.properties` from the repo/package and add to `.gitignore`; purge from history. **Rotate** the Gemini, Maps, and any AdMob credentials. Apply **API key restrictions** (Android app SHA-256 + package for Maps; server-side proxy for Gemini so the key never ships in the client).

F-07 — Chat Messages & Contacts Stored Unencrypted at Rest (Hive) MEDIUM

CVSS 3.1	5.5
CWE	CWE-312 (Cleartext Storage of Sensitive Information)
CVE	N/A (first-party weakness)
OWASP	M9: Insecure Data Storage
Location	lib/core/services/local_storage_helper.dart:109-131 lib/features/chat/auth/service/contact_store_service.dart:19-47

Description. Hive boxes holding chat messages, chat list, conversation metadata, and the imported phonebook are opened with **no** encryptionCipher, storing content in cleartext files under the app documents directory.

```
_messagesBox = await Hive.openBox<String>('messagesBox'); // no encryptionCipher
...
await box.put(_contactKey, jsonEncode(contacts)); // names + phone numbers, plaintext
```

Impact. On a rooted/jailbroken device, via an unencrypted device backup, or through a forensic image, chat content and the full contact list are directly readable. This directly contradicts the app's user-facing "End-to-end encrypted" labelling in the chat/call UI (app_strings.dart), creating a security-and-trust gap.

Remediation. Open Hive boxes with an AES HiveAesCipher whose key is generated once and stored in flutter_secure_storage. Reconcile the E2E marketing claim with the actual at-rest protection (see also F-09/design note).

F-08 — Entire Device Phonebook Uploaded Raw / Unhashed MEDIUM

CVSS 3.1	5.3
CWE	CWE-359 (Exposure of Private Personal Information)
CVE	N/A (privacy weakness)
OWASP	M6: Inadequate Privacy Controls
Location	lib/features/chat/view/contacts/view/contact_list_page.dart:150-174 → chat_view_controller.dart:2939-2969

Description. The full contact list (display names + phone numbers of third parties) is serialized and uploaded to the backend in plaintext for contact discovery, rather than being hashed/tokenized client-side.

```
List<Map<String, dynamic>> rawContacts = contacts.map((c) => {  
  "displayName": c.displayName,  
  "phones": c.phones.map((p) => p.number).toList(),  
}).toList();  
await chatViewController.uploadContacts(formattedContacts);
```

Impact. Non-consenting third parties' PII is transmitted and stored server-side. This raises GDPR/DPDP-Act privacy-compliance exposure and increases breach blast radius.

Remediation. Hash phone numbers (e.g. salted SHA-256 with server-coordinated salt) for discovery; minimize fields uploaded; document a lawful basis and retention policy; provide user opt-out.

F-09 — WebView With Unrestricted JavaScript on Untrusted URLs MEDIUM

CVSS 3.1	6.1
CWE	CWE-79 (XSS), CWE-749 (Exposed Dangerous Method/Feature)
CVE	N/A (first-party weakness)
OWASP	M4: Insufficient Input/Output Validation / M8: Security Misconfiguration
Location	lib/widgets/webview_common.dart:25-27 lib/features/chat/view/widget/chat_video_pip_controller.dart:74-77

Description. `CommonWebView` enables `JavaScriptMode.unrestricted` and loads server-, business-, or user-supplied URLs (business websites, referral links, chat-tapped video links). No `NavigationDelegate/onNavigationRequest` exists anywhere in the app, so the `WebView` can freely navigate to arbitrary origins with JS enabled.

```
controller = WebViewController()
  ..setJavaScriptMode(JavascriptMode.unrestricted)
  ..loadRequest(Uri.parse(urlWithScheme)); // urlLink = business/user-supplied
```

Impact. Attacker-controlled web content (e.g. a malicious "business website" or a link sent in chat) executes JavaScript inside the app's `WebView`. Blast radius is reduced because no JavaScript channel bridges to native code, but session cookies, redirects, and phishing within the trusted app chrome remain viable.

Remediation. Add a `NavigationDelegate` that allowlists schemes/hosts and blocks unexpected navigations; disable JavaScript where not required; consider opening untrusted links in the system browser via `url_launcher` instead of an in-app `WebView`.

F-10 — Deep-Link / Install-Referrer Referral Code Trusted Without Validation

MEDIUM

CVSS 3.1	5.3
CWE	CWE-20 (Improper Input Validation)
CVE	N/A (first-party weakness)
OWASP	M4: Insufficient Input/Output Validation
Location	lib/features/common/onboarding/view/splash_screen.dart:229–234, 275–278

Description. The referralCode query param from any incoming deep link — and the Android Play Install Referrer string — is stored with only a trim/empty check (no format, length, or charset validation) and later injected into an **authenticated** profile-update API (referred_by_code) during onboarding.

```
final referral = uri.queryParameters['referralCode'];
if (referral != null && referral.trim().isEmpty) {
  await SharedPreferences.saveDeferredReferralCode(referral);
}
// later, at signup → updateUserProfileDetails({referred_by_code: saved})
```

Impact. An attacker crafts `https://beapp.in/app/...?referralCode=<attacker>`; a new user who taps it has their signup silently attributed to the attacker → referral/reward fraud. The unbounded value also relies entirely on the server to sanitize.

Remediation. Validate the code against a strict charset/length allowlist at capture time; enforce server-side attribution integrity and anti-abuse checks.

F-11 — Exported Broadcast Receiver Registered Without Permission MEDIUM

CVSS 3.1	5.0
CWE	CWE-926 (Improper Export of Android Application Components)
CVE	N/A (first-party weakness)
OWASP	M8: Security Misconfiguration
Location	android/app/src/main/kotlin/.../MainActivity.kt:249

Description. A dynamic BroadcastReceiver for ACTION_COMPLETE_RIDE is registered with Context.RECEIVER_EXPORTED and no signature/custom permission, so any app on the device can broadcast the action and trigger the "completeRide" method-channel callback into Flutter.

```
val filter = IntentFilter(ACTION_COMPLETE_RIDE)
registerReceiver(pipReceiver, filter, Context.RECEIVER_EXPORTED) // no permission
```

Impact. A malicious co-installed app can spoof ride-completion events, potentially disrupting ride/delivery state or triggering unintended app logic.

Remediation. Use RECEIVER_NOT_EXPORTED (the intent is internal), or protect the receiver with a custom signature-level permission.

F-12 — Hardcoded AES Key & Static IV in Bundled CallKit Package MEDIUM

CVSS 3.1	4.8
CWE	CWE-798, CWE-329 (Not Using Random IV with CBC), CWE-321 (Hard-coded Cryptographic Key)
CVE	N/A (bundled third-party code)
OWASP	M10: Insufficient Cryptography
Location	packages/flutter_callkit_incoming/ios/Classes/StringUtils.swift:13, 21, 41, 45

Description. The vendored CallKit package encrypts the call "handle" (caller name/number) with AES-CBC using a hardcoded key **and** a fixed/static IV reused for every message.

```
func encrypt(encryptionKey: String = "xrBixqjjMhHifSDgSJ804QJYMZ1UHs45",
            iv: String = "lmYSgP3vixDAiBzW") -> String { ... }
```

Impact. A hardcoded key provides zero confidentiality against anyone with the binary; a static IV in CBC leaks equality of plaintexts (identical call handles produce identical ciphertext). Scope is limited to call-metadata obfuscation, but it ships in the app.

Remediation. Prefer a maintained upstream version; if custom crypto is required, use AES-GCM with a per-message random nonce and a key derived/stored securely, never hardcoded.

F-13 — No Certificate / Public-Key Pinning MEDIUM

CVSS 3.1	4.8
CWE	CWE-295 (Improper Certificate Validation)
CVE	N/A (design gap)
OWASP	M5: Insecure Communication
Location	Application-wide (Dio, socket_io_client, raw HttpClient)

Description. No certificate or public-key pinning is implemented; all transport relies solely on the system trust store. Combined with F-04/F-05 (cleartext enabled), there is no defense against a locally-installed rogue CA or transparent proxy. *(Note: no validation is actively disabled — no dangerous `badCertificateCallback` was found — this is a hardening gap, not a bypass.)*

Impact. On devices with an attacker-controlled CA (enterprise MDM, malware, user-installed proxy), API and socket traffic including tokens can be intercepted.

Remediation. Implement pinning for *.beapp.in (Dio `badCertificateCallback` comparing SPKI hashes, or Android `network-security-config` <pin-set>) with a backup pin and a rotation plan.

F-14 — Login OTP Written to Device Logs MEDIUM

CVSS 3.1	4.4
CWE	CWE-532
CVE	N/A (first-party weakness)
OWASP	M9
Location	lib/features/common/auth/views/screens/otp_page_screen.dart:111

Description. The auto-filled SMS login OTP is logged via `debugPrint`. While `debugPrint` is quieter in release, it is not compiled out, and the app-signature hash is also printed nearby.

```
void codeUpdated() {  
  debugPrint("Received OTP: $code");  
}
```

Impact. Time-limited OTP exposure to device logs; combined with other log leaks, weakens the authentication channel.

Remediation. Remove OTP logging entirely; never log authentication factors.

F-15 — Over-Broad Exported Components & Dangerous Permissions MEDIUM

CVSS 3.1	4.6
CWE	CWE-926, CWE-250 (Execution with Unnecessary Privileges)
CVE	N/A (configuration weakness)
OWASP	M8: Security Misconfiguration
Location	android/app/src/main/AndroidManifest.xml

Description. The exported MainActivity declares broad implicit intent filters including SEND/SEND_MULTIPLE for */* and multiple autoVerify app-link hosts, and the app requests a wide, sensitive permission set: ACCESS_BACKGROUND_LOCATION, READ_CONTACTS/WRITE_CONTACTS, SYSTEM_ALERT_WINDOW, RECORD_AUDIO, CAMERA, REQUEST_IGNORE_BATTERY_OPTIMIZATIONS, full-screen intent, etc. (full list in Appendix A).

Impact. A large attack surface for intent spoofing/injection and privacy exposure; over-privilege increases breach impact and invites store-review friction (background location, SAW).

Remediation. Audit each permission against actual feature use and drop unneeded ones; constrain */* SEND filters to required MIME types; validate all inbound intent extras; ensure deep-link hosts have valid Digital Asset Links.

F-16 — Auth Token & Request Bodies Logged (Debug-Gated) MEDIUM

CVSS 3.1	4.0
CWE	CWE-532
CVE	N/A (first-party weakness)
OWASP	M9
Location	lib/core/api/apiService/api_base_helper.dart:122, 135

Description. The request interceptor logs the live Bearer token and full request bodies (which include OTP, contact_no, device tokens). This is wrapped in kDebugMode, so it does not reach release builds, but debug/QA/side-loaded builds leak full credentials.

```
if (kDebugMode) {  
  log("🔑 Authorization: Bearer $authTokenGlobal");  
  log("💎 Body: ${const JsonEncoder.withIndent(' ').convert(options.data)}");  
}
```

Impact. Credential/PII exposure in any non-release build; risky if QA devices are shared or debug APKs circulate.

Remediation. Redact token/secret fields even in debug; use a sanitising logger; ensure debug builds are never distributed externally.

F-17 — Unpinned ("any") Dependency Version Constraints LOW

CVSS 3.1	3.7
CWE	CWE-1104 (Use of Unmaintained/Unpinned Third-Party Components)
CVE	N/A (supply-chain hygiene)
OWASP	M2: Inadequate Supply Chain Security
Location	pubspec.yaml:90-113

Description. Several dependencies use the unbounded any constraint: connectivity_plus: any, http: any, flutter_cache_manager: any. Unpinned versions make builds non-reproducible and can silently pull a compromised or breaking release.

Impact. Supply-chain risk and build drift; a malicious or vulnerable transitive version could be resolved without notice.

Remediation. Pin to caret ranges (e.g. ^x.y.z), commit pubspec.lock, and adopt dependency scanning (dart pub outdated / OSV) in CI.

F-18 — FCM/VoIP Tokens & Payloads Printed to Logs LOW

CVSS 3.1	3.5
CWE	CWE-532
CVE	N/A (first-party weakness)
OWASP	M9
Location	lib/features/common/auth/controller/auth_controller.dart:162; lib/core/services/app_notification.dart:919, 998, 1154, 1227; chat_input_box.dart:1650

Description. FCM push tokens, iOS VoIP/PushKit tokens, and a shared-contact payload are printed without a kDebugMode guard.

Impact. Push-token abuse (targeted push spoofing) and minor PII exposure via logs.

Remediation. Remove or gate these prints; treat push tokens as sensitive.

F-19 — Unsanitized HTML Rendered in WebView (JS Disabled) LOW

CVSS 3.1	3.1
CWE	CWE-79
CVE	N/A (first-party weakness)
OWASP	M4
Location	lib/features/common/jobs/view/resume_preview.dart:22-23

Description. Resume HTML is rendered with `loadHtmlString` and an in-code TODO acknowledging it is unsanitized. JavaScript is explicitly **disabled** here, which mitigates script execution, but HTML/resource injection (external beacons, layout abuse) remains.

```
..setJavaScriptMode(JavaScriptMode.disabled) // Disable JS if not needed
..loadHtmlString(widget.htmlContent); // Consider sanitizing this content first
```

Remediation. Sanitize/whitelist HTML before rendering; block remote resource loads.

F-20 — Live Razorpay & Google Keys Hardcoded in Client LOW

CVSS 3.1	3.1
CWE	CWE-798
CVE	N/A (first-party weakness)
OWASP	M1
Location	lib/environment_config.dart:72; ios/Runner/Info.plist:58; ios/Runner/AppDelegate.swift:16; android/app/google-services.json

Description. A live Razorpay `key_id` (`rzp_live_...`) is committed (commented) and Google Maps/Firebase API keys are embedded in the iOS plist, AppDelegate, and `google-services.json`. Razorpay `key_id` and Firebase keys are *publishable* client identifiers, so exposure is expected — but they must be restricted, and committing the live merchant key in code is poor hygiene.

Impact. Low direct impact (these are not secret keys), but unrestricted Google keys enable quota/billing abuse, and the exposed live merchant id aids reconnaissance.

Remediation. Remove the commented live key; apply Google API key application-restrictions; confirm Firebase Security Rules are locked down (client keys do not grant data access on their own — rules do).

8. Positive Security Observations

The following controls were found to be implemented correctly and should be preserved:

- **Server-side payment verification.** Razorpay orderId/paymentId/signature are forwarded to backend verify-payment endpoints and entitlement is granted only on the server's success — the client never trusts the Razorpay callback alone (`contribution_controller.dart`, `security_deposit_controller.dart`).
- **Secure token storage.** Auth token, user id, mobile, and business id are stored in `flutter_secure_storage` (Android `encryptedSharedPreferences` / iOS `Keychain`, `first_unlock`).
- **WebRTC done right.** ICE/TURN credentials are fetched per-call from the server; no TURN secrets are hardcoded; public STUN-only fallback.
- **Socket auth pattern.** JWT is passed via the Socket.IO auth handshake object over wss, not in a URL query string.
- **Deep-link ID validation.** Routed entity IDs are validated against a strict 24-char hex `ObjectId` regex before navigation; emergency host is exact-matched.
- **Env obfuscation.** API keys are injected via `envied` with `obfuscate:true` rather than plaintext Dart constants.
- **Server-side OTP verification.** OTP is validated by the backend; no client-side OTP comparison/bypass exists.
- **Session hygiene.** A 401 response triggers a full logout, secure-storage wipe, and FCM token rotation.

9. Remediation Roadmap

Priority	Action	Findings	Target
P0	Revoke & rotate all exposed tokens/keys; purge from git history	F-01, F-06, F-20	Immediate (24–48h)
P1	Remove production credential/PII logging	F-02, F-03, F-14, F-16, F-18	This sprint
P1	Re-enable transport security (disable cleartext, scope ATS)	F-04, F-05	This sprint
P2	Encrypt local chat/contact storage; hash contacts before upload	F-07, F-08	Next release
P2	Harden WebView (<code>NavigationDelegate</code> , <code>allowlist</code>); validate deep-link input	F-09, F-10, F-19	Next release
P2	Fix exported receiver; audit permissions & intent filters	F-11, F-15	Next release
P3	Add certificate pinning; replace hardcoded-key crypto; pin dependencies	F-12, F-13, F-17	Backlog / hardening

10. Appendix A — Android Permission Inventory

Declared in android/app/src/main/AndroidManifest.xml. Sensitive/high-risk permissions highlighted for review:

Permission	Risk	Note
ACCESS_BACKGROUND_LOCATION	HIGH	Continuous location tracking; store-review sensitive
ACCESS_FINE_LOCATION / COARSE	MED	Precise location
READ_CONTACTS / WRITE_CONTACTS	HIGH	Feeds contact upload (F-08)
SYSTEM_ALERT_WINDOW	HIGH	Overlay — tapjacking/abuse surface
RECORD_AUDIO / CAMERA	MED	Calls / media capture
READ_MEDIA_IMAGES / VIDEO / EXTERNAL_STORAGE	MED	Media access; legacy storage flag set
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	MED	Background persistence
USE_FULL_SCREEN_INTENT	MED	Incoming-call UI; abuse surface
RECEIVE_BOOT_COMPLETED / FOREGROUND_SERVICE*	LOW	Call/notification persistence
BLUETOOTH / BLUETOOTH_CONNECT / MODIFY_AUDIO_SETTINGS	LOW	Call audio routing
AD_ID	LOW	Advertising ID (Meta Audience Network)

Configuration flags of note: usesCleartextTraffic="true" (F-04), requestLegacyExternalStorage="true", largeHeap="true", exported MainActivity with SEND */* filters (F-15).

11. Appendix B — Tools, References & Standards

Standards Applied

- OWASP MASVS v2 (Mobile Application Security Verification Standard)
- OWASP Mobile Top 10 (2024)
- MITRE CWE (Common Weakness Enumeration)
- CVSS v3.1 severity scoring

Techniques

- Manual white-box source code review (Dart, Kotlin, Swift, Gradle, XML/plist)
- Pattern-based static analysis for secrets, insecure logging, crypto misuse, transport config, IPC exposure

- Configuration & manifest analysis (AndroidManifest, Info.plist, build.gradle, ProGuard, entitlements)
- JWT payload decoding for exposed-token verification

Coverage

2,472 Dart files, Android & iOS native layers, 100+ dependencies, and bundled forked packages were reviewed. Backend behaviour (server-side authorization, JWT signature validation, rate limiting) was inferred from client code and could not be dynamically confirmed; a follow-up DAST/API penetration test against *.beapp.in is recommended.